DNSSEC_Signature_2_working

You are an DNS security expert. Here is an example DNS attack that effects the cache of the resolver to reduce its impact.
Example:
NS cache flush attack
Summary:
Attack owns an auhtoritative server and control example.com. Domain.
The attacker issues A type queries for different unique names under this domain. For example, test1.example.com, test2.example.com test10000.example.com
The attacker's authoritative server responds this queries with around 1500 NS records thus making the DNS reply 65k(the maximum value). Each of these values are cached by the resolver, thus evicting the benign cached records from the resolver's cache causing resolver to requery these domains thus reducing its performance significantly.

I want you to come with another cache flushing attack based on DNSSEC.
utilize your DNSSEC knowledge and check anything can be used against to a resolver for cache flushing purposes.
The aim is to generate 65k DNSSEC responses and have the resolver cache all the data in the answer.

Below is a conceptual DNSSEC-based cache-flushing attack that follows a similar principle to the "NS cache flush attack," but leverages DNSSEC records (DNSKEY, RRSIG, DS, NSEC/NSEC3) to inflate the response sizes up to the 65,535-byte DNS maximum. By filling the resolver's cache with large volumes of DNSSEC data, legitimate cached records are evicted, forcing re-queries and degrading the resolver's performance.

---

1. High-Level Attack Idea

1. **Attacker Zone**
   The attacker controls an authoritative DNS zone, for example:

   ```
   malicious.example.
   ```

2. **DNSSEC-Enabled Zone**
   The attacker signs this zone with DNSSEC using strategies to **maximize** the size of the DNS responses.

3. **Querying Random Subdomains**
   The attacker (or bots under its control) issues queries for **unique or random subdomains** under `malicious.example.`, such as `test1.malicious.example.`, `test2.malicious.example.`, ..., `testN.malicious.example.`

4. **Massive DNSSEC Responses**
   The authoritative server for `malicious.example.` responds with DNSSEC-enabled replies that approach the 65k limit, containing:

   - Multiple large DNSKEY records (e.g., using large RSA or multiple algorithms)
   - Many RRSIG records (one signature per algorithm, per record set)
   - DS records for child zones or sibling zones
   - Potentially extensive NSEC or NSEC3 chains for denial-of-existence

5. **Resolver Cache Bloat**
   Each of these large responses is cached by the resolver. The new data displaces benign or legitimate cached entries, effectively evicting them and forcing re-queries for legitimate domains.

---

2. Technical Details on Inflating DNSSEC Responses

DNSSEC can bloat DNS replies in several ways:

1. **Large DNSKEY Sets**

   - **Multiple Keys**: Sign the zone with multiple DNSKEYs. For instance, 2–3 RSA keys (each 4096 bits) plus an ECDSA key.

- **Large RSA Key Sizes**: 4096-bit RSA keys are not uncommon in testing or misconfigurations. The corresponding DNSKEY record can be *hundreds of bytes* each.
- **Multiple DNSSEC Algorithms**: You can publish keys using RSA, ECDSA, EdDSA, etc. Each demands a unique RRSIG.

2. **Many RRSIG Records**
   - Every RRset in the response can have multiple RRSIGs (one per DNSKEY).
   - If you sign subdomains or even TXT records, each one will come with one or more RRSIGs.
3. **Excessive DS Records**
   - If you control multiple child zones beneath `malicious.example.`, you can publish DS records for each child.
   - Each child's DS record is accompanied by RRSIGs.
4. **NSEC / NSEC3 Chains**
   - When you query random or non-existent subdomains, the server may respond with NSEC or NSEC3 records proving denial of existence.
   - **NSEC3** in particular can be used with:
     - **Multiple iterations**
     - **Large salt values**
     - **Multiple hashed labels**
   - These combinations can result in large sets of NSEC3 records that must be included in the response and cached.

By carefully constructing a zone to include large DNSKEYs, multiple signatures, and possibly an NSEC3 chain, each response can easily approach the 65,535-byte limit permitted by UDP DNS (with EDNS0).

---

3. Attack Steps in Detail

1. **Prepare the Malicious Zone**
   - Create `malicious.example.` with a large number of DNSSEC records.

- Publish multiple DNSKEYs of large sizes (e.g., 4096-bit RSA, 2048-bit RSA, ECDSA, Ed25519, etc.).
- Sign the zone with each key (one RRSIG per key, per RRset).
- Possibly set up sub-delegations, each with DS records pointing to them.

2. **Set EDNS0 Buffer Size**
   - The attacker's authoritative server is configured to allow **large UDP responses** (EDNS0), up to 4096 bytes or the theoretical 65,535-byte maximum.
   - The attacker wants the responses to be large enough to fragment or come very close to the max size.
3. **Generate Queries from Multiple Sources**
   - Use a botnet or multiple vantage points to repeatedly query unique, random hostnames under `malicious.example.`:

     ```python
     test1.malicious.example.
     test2.malicious.example.
     ...
     test10000.malicious.example.
     ```

   - For each query, the attacker's authoritative server includes the full DNSSEC record sets (DNSKEY, RRSIG, DS, etc.) in the Additional or Authority section as necessary.
4. **Resolvers Cache the Large Data**
   - The resolver, respecting DNSSEC, must validate or at least cache all the relevant DNSSEC data.
   - Caching many 60k+ responses drives out other records stored by the resolver, leading to frequent evictions.
5. **Disruption and Performance Degradation**
   - Once benign (legitimate) entries are evicted, subsequent queries for those domains require a fresh lookup.
   - This increases latency and CPU load on the resolver, and potentially on upstream servers.

---

4. Mitigation Thoughts

1. **Configure Cache Limits**
    - Resolvers can limit memory usage for DNSSEC data specifically, evicting old or suspiciously large DNSSEC entries first.
2. **Response Rate Limiting / Aggressive Caching Controls**
    - Some resolvers implement rate-limiting logic or reduce TTLs for responses deemed abnormally large.
3. **DNSSEC Aggressive NSEC/NSEC3 Use**
    - Modern resolvers can use “Aggressive Use of NSEC/NSEC3” to synthesize NXDOMAIN answers without caching all negative records.
    - This can reduce the bloat caused by large NSEC3 chains for negative answers.
4. **Block or Throttle Suspicious Domains**
    - Recursively, operators can implement policies to reject or throttle domains known to generate abnormally large responses.
5. **Limit Number of Concurrent Keys**
    - In real-world setups, zone operators rarely use more than 1–2 DNSKEYs. Multiple large keys or repeated ephemeral signing might be flagged.

---

5. Conclusion

Just like an “NS cache flush attack,” DNSSEC-based attacks aim to inflate the size of DNS replies and evict legitimate cache entries. By leveraging multiple or large DNSKEYs, abundant RRSIG records, and NSEC/NSEC3 proof sets, an attacker can craft 65k DNS responses for each query to unique subdomains. These large records force the resolver to cache numerous DNSSEC RRsets, bloating or flushing out legitimate cached entries and degrading overall resolver performance.

Now , BIND put a max-record limit for any RRSet type as 2. So we need to be more creative to have large responses with 2 records.

The aim is having 65k answers (1 response) not multiple. Focus on any unbounded field that can serve our purpose here.

2/2

Below is one conceptual approach to reach near-65k responses **even when BIND enforces "max 2 records per RRset"**. The core idea is to exploit DNSSEC fields that can be arbitrarily large—in particular, **DNSKEY** (the public key material) and **RRSIG** (the signature blob). By using *extremely* large RSA keys (and thus extremely large signatures), you can inflate a single DNSSEC response up to the DNS 65,535-byte limit—even if you can only have "2 DNSKEYs" and "2 RRSIGs" in a single RRset.

---

1. Key Insight: Exploit the "Unbounded" Parts of DNSSEC

1. **DNSKEY RDATA**
   - DNSKEY RDATA contains the public key bytes.
   - While normal deployments rarely exceed 2048–4096 bits for RSA, *nothing* in the DNS wire-format truly prevents using absurdly large keys (e.g., 32k- or 64k-bit RSA).
   - This translates to thousands of bytes in the public key portion of the DNSKEY record (after Base64 decoding in the wire, it's raw binary).
2. **RRSIG RDATA**
   - RRSIG RDATA contains the signature over an RRset.
   - With a gigantic RSA key, the signature can also be thousands of bytes.
   - You get *one RRSIG per key* used to sign the RRset. So if you have two huge keys, you get two equally huge signatures.

Why These Fields Are "Unbounded"

- **RFC Limits**: DNS itself allows an RDLENGTH up to 65,535 bytes for a single record. There is *no* explicit upper bound for the size of an RSA public key or its signature in the DNSSEC specs. As long as you do not exceed the overall DNS limit of 65535 bytes per record, it's valid (albeit impractical).

- **BIND's "max-record" Setting**: Even if BIND (or another resolver) restricts you to at most 2 DNSKEY records and 2 RRSIG records in an RRset, each of those records can still be *individually massive*.

---

2. Constructing a Single 65k Response with Only 2 Records per RRset

Below is one scenario using your attacker-controlled domain, say `bad.example.`:

1. **Create Two Monstrous RSA Keys**
   - For illustration, choose two RSA keys of "absurd" size—e.g., **32k-bit or 64k-bit**.
     - A 32k-bit RSA key might produce a ~4 KB public key (in raw binary).
     - A 64k-bit RSA key might easily be ~8 KB of public key data.
   - Each key will also generate extremely large signatures, on the order of a few KB.
2. **Publish Exactly Two DNSKEY Records**
   - `DNSKEY #1` – 32k-bit (or 64k-bit) public key.
   - `DNSKEY #2` – Another 32k-bit (or 64k-bit) public key.
   - These two DNSKEY records form the DNSKEY RRset, satisfying the "max 2 records" constraint.
3. **Sign That DNSKEY RRset with Both Keys**
   - Because each DNSKEY signs the DNSKEY RRset, you end up with two RRSIG records:
     - `RRSIG #1` – from Key #1
     - `RRSIG #2` – from Key #2
   - Each RRSIG is huge: the signature data for a 32k-bit key can be 4 KB; for 64k-bit, it can exceed 8 KB.
   - Together, these two RRSIG records can add 8–16 KB of signature data alone.
4. **Single Query → Single Massive Response**
   - A query for `bad.example. IN DNSKEY` forces the authoritative server to return:
     1. DNSKEY #1 (large public key)

2. DNSKEY #2 (large public key)
3. RRSIG(DNSKEY) #1 (large signature)
4. RRSIG(DNSKEY) #2 (large signature)

- In total, these 4 records can easily approach or exceed 60 KB if you size them aggressively. The DNS packet can reach the 65,535-byte maximum (minus overhead).

5. **One RRset → One 65k Response**
   - You're only publishing **two** DNSKEYs and **two** RRSIGs, but each item is itself enormous.
   - BIND's "2 records max per RRset" no longer helps if each record is multi-kilobytes in length; the total still approaches 65k.

(Note: You can similarly bloat DS records or RRSIG(DS) if your zone has a parent relationship, but typically it's easier just to focus on the DNSKEY set at the zone apex.)

---

3. Why This Flushes the Resolver's Cache

- **Each Query = 65k of Cache Data**
  Once a recursive resolver fetches the DNSKEY RRset, it must store all these large records. Even if the TTL is short, for that brief period, the cache holds tens of kilobytes.
- **Repeated Queries to Different Names**
  If you replicate the same idea on multiple subdomains (e.g., `test1.bad.example.`, `test2.bad.example.`, ...) each requiring validation, the resolver must fetch the DNSKEY or negative proof again and store the huge RRSIG data. This quickly consumes cache memory, evicting benign entries.
- **Cache Evictions**
  As the cache saturates, normal entries for other domains get flushed; performance degrades because each eviction forces a re-query later on.

---

4. Potential Mitigations

1. **Enforce Memory or RRset-Size Limits for DNSSEC**
    - Resolvers can impose strict upper bounds on DNSKEY or RRSIG sizes and either truncate or refuse them.
2. **Limit or Refuse Outlandish Key Sizes**
    - Some validating resolvers already treat extremely large keys as misconfigurations and fail validation.
3. **Aggressive TTL Management**
    - Shorten TTL or clamp TTL on suspicious domains to reduce cache retention of huge RRsets.
4. **Policy-Based Blocking**
    - Treat domains producing near-65k DNS responses as malicious and block them entirely.

---

5. Final Takeaway

Even with BIND's "2 records per RRset" rule, *one* DNSKEY RRset containing **two** *monstrously large* keys plus **two** equally large RRSIGs is enough to fill a single DNS response with 65k of data. This "unbounded field" trick (massive public keys and/or signatures) completely circumvents simple record-count limits, enabling an attacker-controlled zone to bloat the resolver cache and degrade its performance with just one huge DNSSEC response.

> In practice, would resolvers accept very large DNSKEYs or signatures?

In principle, DNSSEC (per the RFCs) does not impose a hard upper bound on RSA key sizes or signature sizes—**the 65,535-byte DNS limit** is the only absolute limit at the protocol level. However, **in practice**, most modern resolvers and validators implement *some* combination of the following controls:

1. **Built-in Key/Signature Size Limits**
   Many resolvers include code or configuration parameters that reject DNSKEYs or RRSIGs above a certain size (e.g., rejecting anything above 4096-bit RSA). These checks may be enforced either for performance reasons or based on the assumption that abnormally large keys are effectively a misconfiguration or an attack.
2. **Memory & Cache Constraints**
   Even if a resolver does not explicitly reject large keys, it might run out of memory or trigger other internal limits (e.g., "message size," "maximum memory usage," or "maximum TTL for suspicious data"). Some software also enforces a maximum RRset size, beyond which the record set is truncated or dropped.
3. **Operational / Policy Checks**
   Certain DNS operators or validating resolvers apply local policies, such as:
   - *"Block or fail validation if the DNSKEY size exceeds X bits."*
   - *"Refuse to cache RRsets beyond a certain size."*
     In real deployments, these policies are often used to avoid vulnerabilities like cache-bloat or DOS attacks.
4. **DNSSEC Validation Failures for Outlandish Keys**
   If a key is too large for a resolver to handle properly (e.g., cryptographic libraries fail signature validation due to memory or processing limits), validation breaks and the resolver returns SERVFAIL to the client. Administrators typically see repeated SERVFAIL from a suspicious domain as a sign of misconfiguration or malice.

Bottom Line

- **Technically**: There is no *absolute* RFC-level prohibition on extremely large DNSSEC keys or signatures, other than the 65,535-byte maximum record size.
- **Practically**: Most resolvers *will not* accept or successfully process a 64k-bit RSA key or multi-kilobyte signatures. They either fail validation, exceed operational limits, or are actively configured to reject such data.

So, while using *absurdly large* DNSKEYs or signatures is a *theoretical* vector for creating enormous DNS responses, **real-world resolvers** often implement checks that prevent such oversized records from wreaking havoc on their caches.

If we want to have a resolver to accept a very larger RRSig without actually validating it, how can we achieve this?

Below are several practical ways to have a resolver *accept* (i.e., cache and serve) a very large **RRSIG** (or other DNSSEC data) *without* actually *validating* it. The core idea is that if the resolver is **not** performing DNSSEC validation for that domain (or at all), it will typically still store and return the RRSIG in its cache—because from the resolver's perspective, it is just another piece of DNS data.

---

1. Disable DNSSEC Validation Entirely

One straightforward way is to **turn off DNSSEC validation** altogether in the resolver's configuration. For example, in **BIND** you might do something like:

```perl
options {
    dnssec-validation no;
    // other options...
};
```

With `dnssec-validation no`, BIND will no longer validate signatures at all. It will still *cache* and *serve* DNSSEC RR types (DNSKEY, RRSIG, etc.), but it will not cryptographically verify them. A large RRSIG record would remain in the cache.

Downsides

- This disables validation *globally*, so **all** DNSSEC protection is lost.
- Not realistic for production resolvers, but it's straightforward if your goal is a lab test or demonstration.

---

2. Mark the Domain as "Insecure"

If you do want to keep validation for other domains but skip it for a specific domain you control (e.g., `attacker.example.`), you can tell BIND the domain is **"insecure"** (i.e., has no trust anchor). BIND will then *not attempt validation* for that domain's records, but still cache them.

In BIND's `named.conf` syntax, you can do something like:

```bash
zone "attacker.example" IN {
    type forward;
    forward only;
    forwarders { <authoritative-server-IP>; };
    dnssec-validation no;  // or "dnssec insecure"
};
```

The exact syntax depends on your BIND version, but the effect is that queries for `attacker.example` bypass DNSSEC checks. BIND caches the RRSIG data as normal resource records, ignoring cryptographic validity.

Downsides

- If you're trying to *hide* the fact that you're not validating a particular domain, this is still a configuration-level exception that might be noticed.
- Still more secure than disabling DNSSEC globally, but it allows your target zone to pass in large RRSIGs unchecked.

---

3. Use the "CD" (Checking Disabled) Bit

If you control the **client** side of queries (e.g., you run a custom DNS client or have a botnet that sets DNS flags), you can set the `CD` (Checking Disabled) bit in the DNS query. According to RFC 4035:

> **"Setting the CD bit in the query means the resolver should not perform DNSSEC validation, and just return raw data."**

However, whether BIND (or any particular recursive resolver) *fully* honors the CD bit to skip validation *and still cache the data* can vary. Some resolvers do pass the unvalidated data back

to the client but refuse to *cache* it unless validated. Others will cache anyway. You'd have to check your specific resolver version and configuration.

Downsides

- Not all recursive resolvers honor the CD bit fully for caching. Some will serve you the unvalidated answer but not store it in cache or store it "pending validation."
- This is more relevant when you're making *per-query* decisions rather than a blanket policy for a domain.

---

4. Use a Non-Validating Caching Resolver

If you have the freedom to run your *own* recursive resolver (or have an ISP that never enabled DNSSEC validation in the first place), that resolver will:

1. Cache DNSSEC records (including large RRSIGs).
2. Not attempt cryptographic verification at all.

Some smaller or older deployments never turned on DNSSEC validation by default (particularly in certain public or ISP resolvers). In such environments, big RRSIGs are treated like any other RR in the Additional/Answer sections.

Downsides

- Non-validating resolvers are increasingly rare as DNSSEC adoption grows.
- You have no protection from DNS spoofing or tampering, obviously.

---

5. Practical Considerations & Caveats

1. **Resolver-Specific Size Limits**
   Even if validation is off, large records can hit built-in size limits or memory constraints. Many resolvers limit maximum UDP response size or have an overall "RRset max size" to mitigate resource exhaustion attacks.

2. **Truncation / TCP Fallback**
   If your response is near 65k, it will definitely exceed typical EDNS buffer sizes (~4096 bytes). The resolver will switch to TCP. Some resolvers have separate (often stricter) limits on TCP-based message sizes.
3. **Aggressive Negative Caching**
   If you rely on negative responses with huge `NSEC3` or other records, note that many resolvers use "Aggressive Use of NSEC/NSEC3," generating NXDOMAIN from minimal data. That can limit the bloat you can introduce.
4. **Operational Flags**
   Some resolvers might parse a large RRSIG, notice it does not correspond to a known/valid key length, and simply discard it as malformed or suspicious—even if they are not validating. Behavior can vary depending on the DNS software and version.

Summary

To have a resolver *accept* (cache and serve) a very large RRSIG **without validating it**, you must ensure the resolver is either **not performing validation** on that domain (or at all), or you must exploit the `CD` bit if it causes unvalidated caching (resolver-dependent). Common methods:

1. **Disable DNSSEC validation globally** (e.g., `dnssec-validation no;`).
2. **Mark your specific domain as insecure** or unvalidated in the resolver config.
3. **Use the CD bit** (if the resolver's caching behavior allows storing unvalidated data).
4. **Run or pick a non-validating resolver** that does not do DNSSEC checks to begin with.

All of these approaches circumvent cryptographic checks, letting you push large RRSIG data into the cache, but each has drawbacks—either from a security or operational standpoint.

> Let's say our domains is DNSSEC enabled, checking is not disabled, resolver is validating resolver but we still want to have our very large RRSig to be cached without any validation. Is there any way to this? Check DNSSEC documentation

Below is one practical trick (and the closest thing in the DNSSEC specs) to **publish DNSSEC data** (including very large RRSIGs) but have a **validating resolver** effectively *bypass* cryptographic checks. The key is to ensure that, from the resolver's point of view, **no secure delegation** exists for your zone—so the zone is treated as "insecure," and the resolver does not attempt to validate the signatures. The large RRSIGs then get cached as mere "additional" DNS records.

---

1. Publish DNSSEC Records **Without** a Parent DS

1. **Sign your zone**: You create a DNSKEY, generate RRSIGs, etc. So your zone is "DNSSEC-enabled" in that it publishes keys and signatures.
2. **Do not publish a DS record in the parent**: In other words, when you register `example.com` (or any domain at the TLD), you simply **do not** upload the DS to the TLD registrar.

Effect in DNSSEC Terms

- Since the parent zone (e.g., the TLD `.com`) has **no DS** pointing to your zone's DNSKEY, the resolver sees **no secure delegation**.
- Per RFC 4033 and RFC 4035, a zone without a DS in the parent is considered "insecure," even if it publishes DNSKEY and RRSIG.
- A validating resolver **only validates** if it has a chain of trust from a known trust anchor down to the child zone's DNSKEY via DS records. No DS → no chain → "insecure."

Consequence

- The resolver **does not** perform cryptographic checks on your large RRSIGs (because from its perspective, there is nothing to validate against—no secure path).
- The resolver **still caches** your DNSKEY, RRSIG, etc. as part of the normal DNS data. It just doesn't treat them as authoritative proofs.
- Your large RRSIG goes into the cache, unvalidated.

---

2. Why This Works

DNSSEC resolvers follow a strict hierarchy:

1. **Root / TLD** publishes a DS for your domain if you opt in to the chain of trust.
2. The DS in the parent references your domain's DNSKEY.
3. If no DS is present, the resolver marks the zone "insecure."

Under the RFCs:

- An "insecure" zone can still serve DNSKEY and RRSIG records.
- The validating resolver does **not** attempt to authenticate them, because there's no DS bridging the parent to the child.

Hence, you can publish enormous DNSKEYs, RRSIGs, or other DNSSEC records, and they will appear in the resolver's cache as normal RR data. The resolver does not discard them for failing cryptographic checks, because it never tries to validate them in the first place.

---

3. Example Walkthrough

1. **Sign** `bigrrsig.example.com`
   - You create a gigantic RSA key (say 8192 bits or more) or multiple large keys.
   - Generate an equally large RRSIG covering the zone's apex or other RRsets.
2. **No DS at the** `.com` **level**
   - In your domain registrar's DNSSEC settings, you simply do **not** upload the DS.
   - The `.com` zone (the parent) thus has no knowledge of your DNSKEY, so from a trust perspective, `bigrrsig.example.com` is insecure.
3. **A Validating Resolver Queries**
   - When a validating resolver (e.g., BIND with default DNSSEC validation enabled) queries for your zone's records, it receives your DNSKEY and RRSIG.
   - Because there is *no DS* in `.com`, the resolver says: "No chain of trust → the zone is insecure → do not validate."

- The resolver **caches** your RRSIG as normal data.

---

4. Caveats & Practical Limits

1. **Resolver Size Limits Still Apply**
   - Even if the resolver doesn't validate the RRSIG, it might still reject or truncate extremely large responses (e.g., near 65k).
   - Many resolvers implement "maximum RRset size" or "maximum message size" to defend against cache-bloat attacks.
2. **No "AD" Bit**
   - Because your zone is insecure, validating resolvers will never set the `AD` (Authenticated Data) bit in responses for your domain.
   - The data is effectively "unvalidated, just cached."
3. **Parent Might Publish a Bogus DS**
   - If someone erroneously sets a DS in the parent, it could turn your zone from "insecure" into "bogus," and resolvers would fail to resolve your domain.
   - So you must ensure *no DS record* is present to remain insecure.
4. **Not "Partial" DS**
   - A partial DS is not recognized in DNSSEC—either you have a DS that matches your DNSKEY or no DS at all. If the DS is mismatched, your zone is "bogus," and validation fails completely.

---

5. Summary

- **Question**: "How can I have a validating resolver cache a large RRSIG without validating it?"
- **Answer**: Publish DNSSEC records (DNSKEY, RRSIG) in your zone but **do not** place a DS record in the parent zone. From the resolver's perspective, your zone is "insecure," so it does not perform cryptographic checks. It still caches the big RRSIG as normal DNS data.

This is the main (and standards-compliant) way to serve “DNSSEC-like” data to a validating resolver while bypassing actual validation.

No , the domain will have a DS record and it is secure. Be more creative

Below is one **“creative”** technique to publish *massively large* RRSIG records under a **secure (DNSSEC-enabled) domain with a valid DS**—while effectively **skipping** validation *for the huge signature itself*. The trick relies on **publishing multiple DNSKEYs and multiple RRSIGs** under different DNSSEC algorithms—at least one of which is **recognized** (so the zone validates overall) and **one (or more)** that is **unknown or unsupported** by the resolver (so that particular RRSIG is never actually validated). The resolver still treats the zone as **secure**, but it will **cache** the unvalidated (and potentially huge) RRSIG for the unknown algorithm.

---

1. Background: Multiple DNSSEC Algorithms in One Zone

DNSSEC allows a zone to be signed simultaneously by **multiple DNSKEYs**, each potentially using a **different** cryptographic algorithm (e.g., RSA/SHA-256, ECDSA, Ed25519, etc.). For each DNSKEY, you generate a corresponding **RRSIG** over each RRset.

- **RFC 6840**, Section 5.2, clarifies that if **at least one** recognized algorithm/path can be validated successfully, the zone is considered **secure**—even if other algorithms or signatures are **unknown** or **unsupported** by the validator.

Put differently:

> *If a resolver finds a valid chain of trust via any one known algorithm, it marks the data secure. Other unrecognized or unsupported RRSIGs do not cause the zone to become bogus.*

---

2. High-Level Attack/Technique

1. **Keep the Domain Fully Secure**
    - You have a valid **DS** in the parent zone.

- At least **one** of your DNSKEYs uses a mainstream algorithm (e.g., RSA/SHA-256) that the resolver recognizes.
- That legitimate key signs your zone normally, producing standard-sized RRSIGs that fully validate.

2. **Add an Unknown / Unsupported Algorithm**
   - You register a **custom or "private-use" DNSSEC algorithm code** (e.g., something the validator doesn't implement).
   - For example, choose an algorithm number in the "Reserved for Private Use" range, or a newly allocated code that few resolvers support yet.
   - Publish a **second** DNSKEY of that unknown algorithm with an **absurdly large** key size (e.g., 16k–64k bits if RSA-based).
3. **Generate a Gargantuan RRSIG**
   - Using that huge unknown-algo DNSKEY, generate an **enormous** signature (the RRSIG).
   - Because RSA signature size scales roughly with the key size, you can create multi-kilobyte (or near-65k) RRSIG RDATA.
4. **Zone Has Two RRSIGs per RRset**
   - **RRSIG #1 (Recognized Algo)**: e.g., `RSASHA256` or `ECDSA`. The resolver *can* and *will* validate this.
   - **RRSIG #2 (Unknown/Unsupported Algo)**: e.g., "Algorithm 253/254" or some not-yet-implemented algorithm ID. This one has a **massive** signature field.
5. **Resolver Behavior**
   - Because there is a valid DS in the parent and at least one RRSIG from a recognized algorithm, the zone **passes** DNSSEC validation overall.
   - The "unknown algorithm" RRSIG is effectively **ignored** for validation. The resolver sees it, doesn't know how to validate it, but does **not** treat the zone as bogus.
   - Typically, the resolver **caches** *all* RRs returned from the authoritative server—even those it did not use in successful validation—unless it has specific code to discard them.
   - Result: The monstrous RRSIG is stored in the resolver's cache.

Hence, you have a **fully secure domain** (DS present, recognized key is valid) while shipping a **huge** unvalidated RRSIG that lands in the resolver's cache.

---

3. Why This Works

- **DNSSEC "At-Least-One" Validation Model**
  Under DNSSEC standards, if any **one** recognized crypto algorithm's signature is valid, the resolver **must** treat the zone as **secure**. An unrecognized signature *cannot* make an otherwise valid zone become bogus.
- **Caching of Extra RRsets**
  By default, most resolvers store **all** data in a response if it isn't obviously bogus or out-of-bailiwick. That includes extra RRSIGs for algorithms they don't handle.

Because the zone is successfully validated by the recognized signature, there's no reason for the resolver to discard the "extra" RRSIG. The large RRSIG from the unknown algorithm simply goes unvalidated, but still ends up in the cache.

---

4. Practical Steps

1. **Obtain/Define a "Private" DNSSEC Algorithm**
   - For testing or malicious demonstration, pick an algorithm number in the "private use" range (e.g., RFC 6014).
   - Write or modify a signing tool (like `ldns-signzone`, `dnssec-signzone`, etc.) to generate a "fake" signature using your large key.
2. **Keep a Normal Algorithm**
   - Use a mainstream algorithm (e.g., RSA/SHA-256, ECDSA P-256, etc.) for the real chain of trust.
   - Publish that DNSKEY in the parent's DS record so the domain is recognized as "secure."
3. **Maximize the "Unknown" RRSIG**

- Create an RSA key of *extreme size* (16k-bit or more).
- Generate a signature with it (the “private” algorithm). The signature can be thousands of bytes.
- Because an RRSIG’s signature field is unbounded (up to the overall DNS 65,535-byte limit), you can approach the maximum packet size.

4. **Push That Huge RRSIG in Your Zone**
   - When you sign each RRset, produce *two* RRSIGs:
     1. The normal recognized algorithm RRSIG.
     2. The massive unknown algorithm RRSIG.
   - The authority server returns both in the Answer or Authority section as appropriate.
5. **Resolver Fetches and Caches**
   - During resolution, the resolver sees the DS from the parent, downloads your zone’s DNSKEYs, sees one recognized key (and one unknown), checks the recognized signature → success.
   - The unknown signature is left unvalidated, but not thrown away. It’s simply an extra record.
   - In many implementations (BIND, Unbound, etc.), that extra RRSIG is then stored in the cache for its TTL.

---

5. Potential Obstacles & Considerations

1. **Implementation Differences**
   - Some resolvers might discard RRSIGs that use unknown algorithms.
   - Others will keep them around as they do no harm if at least one valid signature was found. Behavior can vary by version and configuration.
2. **Message Size / Fragmentation**
   - A near-65k response will definitely require TCP fallback, which might trigger more stringent checks or rate limits.

- Some resolvers or middleboxes may drop or truncate extremely large DNS responses.

3. **Possible "Bogus" If the Resolver *Does* Know the Algorithm**
   - If the resolver unexpectedly supports your "unknown" algorithm but cannot properly validate the signature (e.g., key too large?), it might mark the zone "bogus." This depends on how strictly the code tries to do the cryptography.
   - Typically, though, truly "unknown" or "private" algorithm numbers are simply ignored.
4. **Parent DS**
   - Make sure the DS in the parent references only your *recognized* DNSKEY (the standard algorithm). Do not attempt to reference the unknown key.

Final Summary

- **Goal**: Force a validating resolver (with a real DS chain of trust) to cache a huge RRSIG *unvalidated*.
- **Method**:
  1. Sign the zone with **two** algorithms: one standard (to ensure actual validation) and one **private/unrecognized** (carrying a gigantic signature).
  2. Publish both RRSIGs in each answer.
  3. The resolver *validates* the known algorithm's RRSIG, treats the zone as secure, **does not** discard the unknown RRSIG, and **caches** the huge signature anyway.

In effect, you have a *secure*, DS-enabled domain that **skips** validation on the giant RRSIG while still caching it—achieving the "large, unvalidated data in a secure zone" outcome.

Does resolver need to support the unknown algorithm for caching? because the resolver is not validating it.

2/2

Short Answer:
No. A validating resolver does **not** need to *support* (i.e., understand or validate) the unknown algorithm in order to *cache* that RRSIG. As long as **one recognized algorithm** validates the zone, the resolver marks the zone as "secure" and retains **all** the associated RRSIGs (including the unknown-algorithm one) in its cache—even though it never actually validates that unknown signature.

Why the Resolver Caches an "Unknown-Algorithm" RRSIG

1. **DNSSEC's "At-Least-One" Validation Rule**
   - Under DNSSEC (per RFC 6840, Section 5.2), if the resolver can successfully validate an RRset with **any one** recognized algorithm, the data is considered *secure*.
   - Any additional RRSIGs using algorithms the resolver does not recognize **do not** cause the zone to become *bogus*. They are simply ignored for validation purposes.
2. **Resolvers Typically Cache *All* RR Data in a Validated Zone**
   - Once the resolver determines the zone is *secure* (because it validated at least one known signature), it normally caches **all** the records in the response (subject to any local policy or size limit).
   - This includes any extra RRSIGs (even if the algorithm is unknown or unsupported).
3. **No Need to Parse or Verify the Unknown Signature**
   - The resolver only needs to verify the RRSIG(s) for the algorithm(s) it *does* support.
   - For the unknown algorithm's RRSIG, the resolver does not even attempt validation. It treats it as additional data, not affecting the overall "secure" status of the zone.
4. **Result**
   - The large, unknown-algorithm RRSIG record is *stored* in the cache with the rest of the zone data, even though it was never validated.

What If the Resolver *Did* "Support" the Unknown Algorithm?

- If the resolver happens to *partially* support or recognize that algorithm but **cannot** successfully validate the signature (for instance, due to an impossibly large key or incomplete cryptographic support), there is a risk it could mark the zone as "bogus."
- To ensure that big RRSIG does *not* break validation, you want an **algorithm code** that the resolver truly does **not** implement. Then, by the DNSSEC spec, the resolver must ignore that signature rather than fail on it.

Practical Caveats

1. **Some Resolvers May Strip or Reject Unknown RRSIGs**
   - Certain resolvers (or custom patches) might have a policy to discard RRSIGs they cannot validate. This is not standard behavior but can happen in hardened or heavily customized DNS setups.
2. **Message Size & Fragmentation**
   - A giant RRSIG could push the DNS response toward 65k bytes. The resolver may fall back to TCP, or large responses might be dropped by firewalls/middleboxes if they exceed certain limits.
3. **Chain of Trust Must Be Valid for at Least One Key**
   - Remember that your domain must publish a DS record that matches the *recognized* DNSKEY. That key's RRSIG ensures the zone is accepted as secure.

Summary

A validating resolver does **not** need to *support or validate* an unknown DNSSEC algorithm to store its RRSIG in cache. As long as the zone successfully validates under a known, recognized algorithm, the resolver treats the zone as "secure" and retains **all** the RRSIGs

(including the big “unknown-algo” one) in its cache. This is exactly how you can publish *very large* RRSIGs in a **secure** domain while bypassing validation for those oversized signatures.

So can this second signature be a bogus and made up one?

Short Answer:
Yes. If the second signature uses an **unsupported or “private-use” DNSSEC algorithm**, the resolver will not attempt to validate it. From the resolver’s perspective, that signature is simply “unrecognized data,” so you could fill it with essentially **any** bytes. As long as the zone **does** have at least one **valid, recognized** signature/algorithm, the domain will still be considered “secure,” and the **bogus** (or random) second signature will simply be cached without validation.

Why “Bogus” or Random Data Can Be Used

1. **DNSSEC’s Multi-Algorithm Rule**
    - A zone can publish multiple DNSKEYs, each under a different algorithm.
    - If **one** (recognized) algorithm validates successfully, the zone is marked “secure.” The resolver ignores any unknown or unsupported algorithms—it does **not** cause the zone to become “bogus.”
2. **Unknown Algorithms Are Ignored**
    - Per the DNSSEC specifications (RFC 6840, Section 5.2), if a resolver doesn’t implement or recognize the algorithm number, it **does not** validate that signature.
    - “Bogus” in DNSSEC terms usually means “recognized but cryptographically invalid.” But if the algorithm is unknown, the resolver doesn’t even try to validate it, so it never gets flagged as bogus.
3. **Resolver Caches All RRSIGs from a Secure Zone**
    - Once the zone is deemed “secure” by at least one valid signature, the resolver typically caches **all** RRSIGs included in the response.
    - That includes the second “unknown-algo” RRSIG—even if it’s just random data.
4. **No Impact on Validation**

- Because the zone **does** have a DS at the parent and a valid signature from a recognized algorithm, the chain of trust is intact.
- The unknown-algo signature does not matter to the resolver's final "secure" vs. "bogus" decision.

Practical Example

1. **One "Legit" Algorithm**
   - Suppose you have a DNSKEY using RSA/SHA-256 (algorithm 8) or ECDSA (algorithm 13), which is widely recognized.
   - You configure your parent DS to match this key, so a validating resolver can build a chain of trust and confirm the domain is properly signed.
2. **One "Fake" or Unknown Algorithm**
   - You add a second DNSKEY that uses a "private-use" or not-yet-implemented algorithm number (e.g., 253 or 254).
   - You generate a signature (RRSIG) for that second key, but in reality, you can stick **arbitrary** or random data in the signature field—there is no standard validation because the resolver doesn't know how.
3. **Resolver Behavior**
   - Upon lookup, the resolver sees two RRSIGs.
     - **RRSIG #1** (RSA/SHA-256): Validates fine → zone is "secure."
     - **RRSIG #2** (unknown algorithm): Skipped for validation because the algorithm is not recognized.
   - The resolver proceeds to cache **both** RRSIG #1 and #2. The second signature is effectively "unvalidated junk," yet still stored.

Caveats

1. **Some Resolvers May Discard Unknown RRSIGs**

- Not all resolvers behave identically. Some might drop (not cache) RRSIGs with unknown algorithms. Others might keep them as normal resource records.

2. **Size/Message Limits**
   - If you put truly massive or random data in the second RRSIG, you risk hitting message-size limits or being truncated by middleboxes.
   - You may need TCP fallback, which some resolvers or firewalls treat more strictly.
3. **Ensure the Recognized Algorithm Truly Validates**
   - If your “legit” signature fails, the zone is bogus overall. The entire zone (including your second signature) becomes irrelevant to resolvers.

Final Takeaway

- **Yes**, that second signature can be entirely “bogus” or random **as long as** it uses an **unknown or unsupported** DNSSEC algorithm.
- The resolver will **only** validate the known algorithm’s signature. If that one is good, the zone remains secure, and the **junk** signature is simply cached without scrutiny.